PROJECT REPORT

SECURE INLINE COMMUNICATION CHANNEL

Design, implementation, and evaluation

STUDENT	Anahit Darbinyan
SUPERVISOR	Mamikon Mirzoyan
INSTITUTION	French University in Armenia
COURSE / DEPARTMENT	Networks 3

Yerevan 2026

Table of contents

Table of contents **2**
1. Introduction **3**
1.1 Background 3
1.2 Problem Statement 3
1.3 Aim and Objectives 4
1.4 Scope 4
2. System Architecture and Design **5**
2.1 Implemented Architecture 5
2.2 Component Responsibilities 5
2.3 Protected Packet Format 5
2.4 Session Establishment 6
2.5 Packet-Processing Approach 7
2.6 Test Topology 7
3. Technologies and Security Concepts **8**
3.1 TCP/IP and IPv4 8
3.2 X25519 Key Exchange and Peer Authentication 8
3.3 HKDF-SHA-256 8
3.4 AES-256-GCM and Nonce Management 8
3.5 Linux NFQUEUE 9
4. Implementation **10**
4.1 Development Environment 10
4.2 Project Structure 10
4.3 Cryptographic Implementation 11
4.4 Handshake and Authentication 11
4.5 NFQUEUE Packet Handler 11
4.6 Packet Modification and TCP Considerations 11
4.7 Anti-Replay Protection 12
5. Testing and Results **13**
5.1 Cryptographic Component Tests 13
5.2 Device-to-Device Session and Replay Tests 15
5.3 NFQUEUE Tests 18
5.4 Final System: Unit Tests 22
5.5 Final System: End-to-End Test 23
5.6 Requirements Summary 25
6. Security Analysis and Limitations **26**
7. Conclusion **27**
Appendix A. Build and Execution Commands **28**
Appendix B. Additional Test Output **29**

1. Introduction

1.1 Background

The exchange of information over computer networks has become an essential part of modern communication systems. Data is frequently transmitted between computers and other network devices through Internet Protocol (IP) networks. Although these networks provide an efficient way to exchange information, transmitted data may be exposed to various security threats, particularly when communication is not properly protected.

One of the main concerns in network communication is the possibility of unauthorized access to transmitted information. Attackers may intercept network traffic, monitor exchanged messages, or attempt to modify data while it is being transmitted between communicating devices. Such activities can compromise the confidentiality and integrity of information.

Cryptographic techniques play an important role in protecting data during transmission. Encryption ensures that information cannot be easily understood by unauthorized parties, while authentication mechanisms help verify the integrity and authenticity of transmitted data. Secure communication systems commonly combine key exchange mechanisms, key derivation functions, and authenticated encryption algorithms to establish and maintain protected communication sessions.

This project implements a secure inline communication channel for TCP over IPv4. Two Linux-based devices are placed between two hosts. They establish a shared session key with an authenticated X25519 Diffie–Hellman exchange, derive the key with HKDF-SHA-256, and then protect selected TCP packets with AES-256-GCM. The packets are intercepted and rewritten in user space with Linux NFQUEUE, so that the two end hosts can keep using ordinary, unmodified TCP applications.

1.2 Problem Statement

Standard TCP provides reliable delivery, but it does not by itself guarantee the confidentiality or the cryptographic integrity of the transmitted application data. The problem addressed in this project is how to protect TCP traffic between two hosts, PC1 and PC2, without changing either host, by placing two security devices, Device1 and Device2, in the path: PC1 → Device1 → Device2 → PC2.

The devices must process selected TCP/IPv4 traffic and establish a secure session before any protected data is sent. For every protected TCP packet, only the TCP payload is encrypted with AES-GCM; the original IP and TCP headers are preserved and the AES-GCM authentication tag is appended to the encrypted payload. The modified packet is forwarded directly to the peer device, without UDP encapsulation and without a custom protocol header. The receiving device must verify the authentication tag, decrypt the payload, restore the original packet structure, recalculate the IP and TCP checksums, and forward the packet to the destination host. A unique AES-GCM nonce must be used for every encrypted packet within a session.

Working with packets inside a TCP stream raises specific technical challenges: the packet length changes, the IP and TCP checksums become invalid, TCP sequence and acknowledgment numbers must remain consistent for the end hosts, retransmitted segments must not reuse a

nonce, and the extra bytes must still fit into the link MTU. These challenges are addressed in Chapters 2 and 4.

1.3 Aim and Objectives

The aim of this project is to design, implement, and evaluate a Linux-based prototype of a secure inline communication channel between two hosts using two intermediate devices, authenticated key establishment, and AES-GCM encryption of TCP payloads. The following objectives were defined and achieved:

- Design a communication architecture with four components: PC1, Device1, Device2, and PC2.
- Establish a shared session key between Device1 and Device2 with an X25519 Diffie–Hellman exchange and derive it with HKDF-SHA-256.
- Authenticate both devices during the handshake (pre-shared key proofs over the handshake transcript) so that a man-in-the-middle cannot take part in the exchange.
- Encrypt only the TCP payload with AES-256-GCM, keep the IP and TCP headers, and append the 16-byte authentication tag.
- Intercept selected IPv4 TCP packets in user space with Linux NFQUEUE, rewrite them, and reinject them without UDP encapsulation or a custom header.
- On the receiving device, verify the tag, decrypt, restore the original packet, recalculate the IP and TCP checksums, and forward the packet.
- Guarantee a unique nonce for every encrypted packet in a session and reject replayed, modified, or forged packets.
- Evaluate the prototype with cryptographic tests, packet-level unit tests, and an end-to-end test over four network namespaces.

1.4 Scope

The project covers the complete data path PC1 → Device1 → Device2 → PC2 for TCP over IPv4. The cryptographic part includes X25519 key agreement, HMAC-SHA-256 handshake authentication, HKDF-SHA-256 key derivation, and AES-256-GCM packet protection with a per-packet counter nonce and a sliding anti-replay window. The networking part includes the NFQUEUE-based packet processor, the payload replacement, the packet length update, and the IP/TCP checksum recalculation.

The system was developed and tested in a Kali Linux virtual machine. The four hosts are emulated with Linux network namespaces connected by virtual Ethernet pairs. This is a realistic test of the packet path, but it is not a test across physical machines.

Several aspects are intentionally outside the scope of this prototype and are discussed in Chapters 6 and 7: only the PC1 → PC2 direction is protected (return traffic is forwarded unchanged); the pre-shared authentication key is hard-coded for demonstration; a session uses one key without re-keying; IPv6 is not supported; and fragmented IP packets are dropped rather than processed.

2. System Architecture and Design

2.1 Implemented Architecture

The system consists of four components connected in the following order: **PC1 → Device1 → Device2 → PC2**. PC1 generates ordinary TCP traffic towards PC2. Device1 sits in the path as an IP router; for the selected traffic it encrypts the TCP payload. Device2 is the next router; it authenticates and decrypts the payload and forwards the original packet to PC2. Neither PC1 nor PC2 is aware of the protection. Figure 1 shows the topology used in the tests, including the IP addresses.

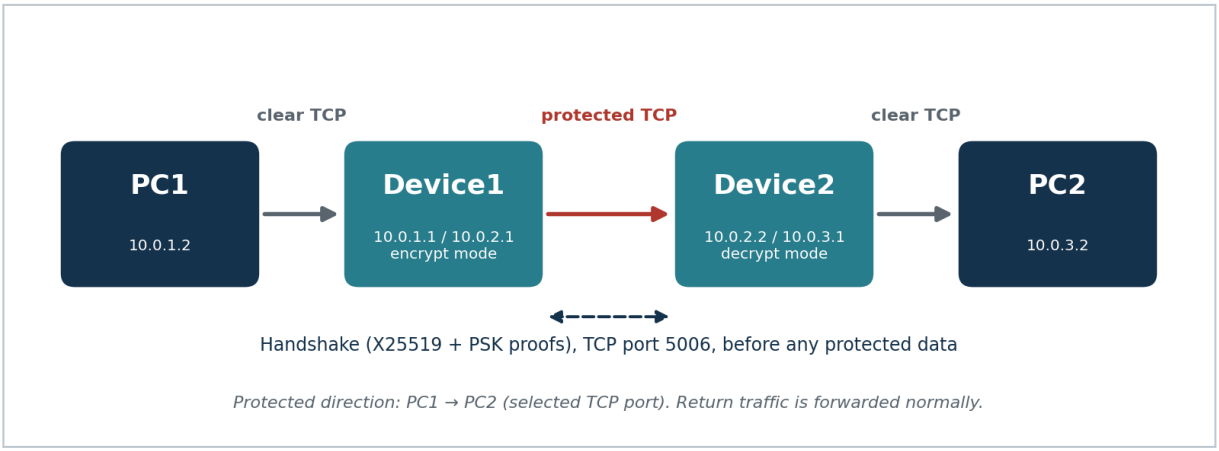


Figure 1. *Test topology: four network namespaces connected by virtual Ethernet links. The link between Device1 and Device2 carries the protected packets.*

2.2 Component Responsibilities

Table 1. *Component responsibilities in the implemented system.*

Component	Responsibility	Implementation
PC1	Generates TCP traffic and sends it towards PC2 (default route via Device1).	Network namespace pc1 (10.0.1.2); Python/netcat test clients.
Device1	Runs the handshake as client, then intercepts selected PC1 → PC2 packets and encrypts the TCP payload with AES-256-GCM.	`device --mode encrypt`, NFQUEUE 2, `pp_encrypt_packet()`.
Device2	Runs the handshake as server, verifies the authentication tag, decrypts the payload, restores the packet, and forwards it to PC2.	`device --mode decrypt`, NFQUEUE 3, `pp_decrypt_packet()`.
PC2	Receives the original, restored application data.	Network namespace pc2 (10.0.3.2); Python/netcat test server.

2.3 Protected Packet Format

Each selected TCP packet that carries data is transformed as shown in Figure 2. The IPv4 and TCP headers are kept. The TCP payload of N bytes is replaced by an 8-byte packet counter, the N-byte AES-256-GCM ciphertext, and the 16-byte authentication tag, so every protected packet grows by exactly 24 bytes. Only the IP Total Length field and the IP and TCP checksums

change. The packet remains an ordinary TCP/IPv4 packet with the same source and destination addresses and ports; there is no UDP encapsulation and no extra protocol header.

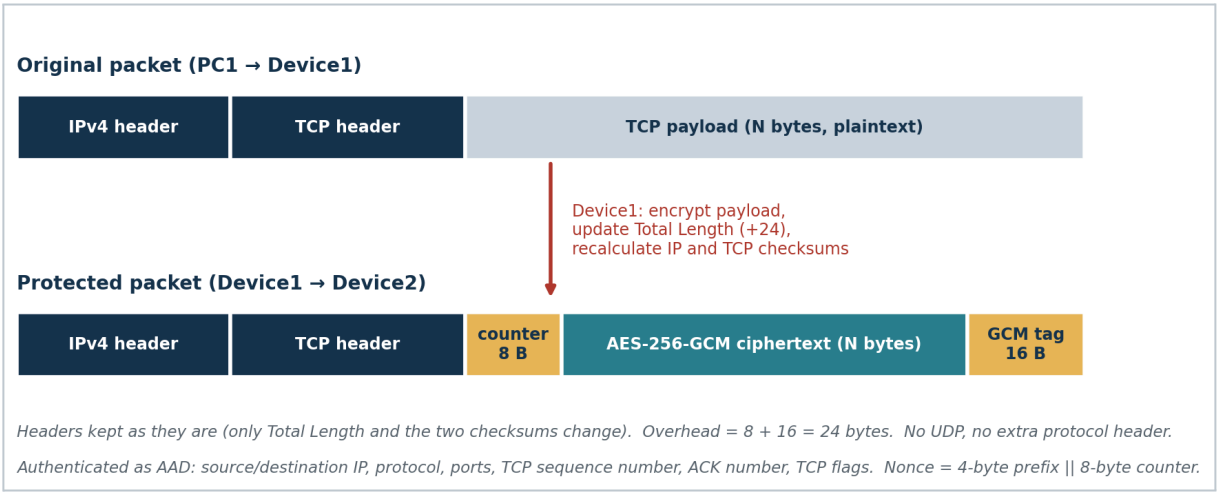


Figure 2. *Packet format before and after protection. Only the payload area is transformed.*

The 96-bit AES-GCM nonce is built from a fixed 4-byte prefix and the 64-bit counter: nonce = prefix ‖ counter. Only the counter is transmitted, because the receiver already knows the prefix. The following header fields are authenticated as additional authenticated data (AAD, 22 bytes): source and destination IP address, protocol, source and destination port, TCP sequence number, TCP acknowledgment number, and the TCP flags. The TTL and the two checksums are intentionally not part of the AAD, because routers decrement the TTL and recalculate checksums. Packets with an empty TCP payload (SYN, pure ACK, FIN) carry no application data and are forwarded unchanged.

2.4 Session Establishment

Before any protected data is sent, Device1 and Device2 run a handshake over a normal TCP connection on port 5006 (Figure 3). Device1 acts as client and Device2 as server. Both devices generate an ephemeral X25519 key pair, so a new session key is obtained for every run. Device2 additionally supplies a fresh random 16-byte salt. Both devices then prove knowledge of a pre-shared key (PSK) by sending an HMAC-SHA-256 proof computed over a transcript that contains both public keys and the salt. Because the transcript is bound to the proof, an active attacker who substitutes a public key cannot produce a valid proof. After both proofs are verified, each device derives the AES-256 session key with HKDF-SHA-256 using the shared secret, the salt, and the context label `device1-to-device2`. A device does not open its NFQUEUE until the handshake has succeeded.

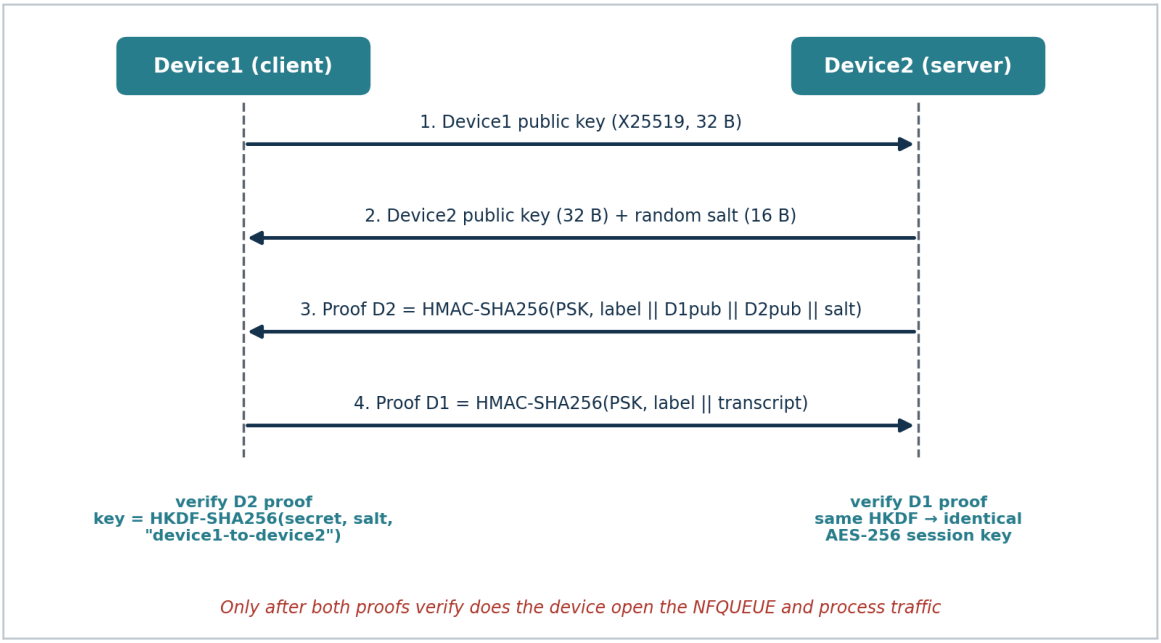


Figure 3. *Authenticated handshake between Device1 and Device2 (X25519, HMAC-SHA-256 proofs, HKDF-SHA-256).*

2.5 Packet-Processing Approach

Linux NFQUEUE passes selected packets from the kernel to a user-space program, which returns a verdict. In this project, an iptables rule in the FORWARD chain of each device sends the selected traffic to a queue: TCP packets from 10.0.1.0/24 to 10.0.3.0/24 with the chosen destination port (5001 in the tests). Device1 reads queue 2 and Device2 reads queue 3. Traffic that does not match the rule, such as the return traffic from PC2 and the handshake between the devices, is forwarded normally.

For each queued packet the program copies the packet into a work buffer, encrypts or decrypts it, and returns `NF_ACCEPT` together with the modified packet and its new length, so the kernel forwards the rewritten packet. If a packet cannot be processed correctly (authentication failure, replay, malformed or fragmented packet, exhausted nonce counter) the verdict is `NF_DROP`. The system therefore **fails closed**: selected traffic is never forwarded in clear text because of an error, and if a device is not running, the NFQUEUE rule drops the matching packets.

2.6 Test Topology

The topology is created by the script `scripts/setup_topology.sh`. It creates four namespaces (pc1, d1, d2, pc2), three virtual Ethernet pairs, the addresses shown in Figure 1, and the routes. IP forwarding is enabled in d1 and d2. Because the protected packet is 24 bytes longer than the original, the MTU of the PC interfaces is set to 1476 bytes so that the largest protected packet is exactly 1500 bytes on the link between the devices. TCP, generic segmentation, and receive offloads are disabled on the virtual interfaces so that NFQUEUE always sees normal, MTU-sized packets.

3. Technologies and Security Concepts

3.1 TCP/IP and IPv4

The Internet Protocol version 4 (IPv4) is responsible for addressing and routing packets between devices across a network. An IPv4 packet contains a header with information such as source and destination IP addresses, total length, protocol type, and a header checksum. The Transmission Control Protocol (TCP) provides reliable, connection-oriented communication between applications. Its header contains source and destination port numbers, sequence numbers, acknowledgment numbers, flags, and a checksum that covers the TCP header, the payload, and a pseudo-header derived from the IP addresses.

TCP carries application data as a continuous byte stream. The sequence number of a segment counts bytes of this stream. This is important for the project: as long as the receiving device restores each segment to its original length, the sequence and acknowledgment numbers seen by PC1 and PC2 stay consistent and the endpoints need no special handling.

3.2 X25519 Key Exchange and Peer Authentication

X25519 is an elliptic-curve Diffie–Hellman function (RFC 7748). Each party generates a private key and a public key, exchanges the public keys, and combines the received public key with its own private key to obtain the same shared secret without ever sending that secret over the network. In this project fresh (ephemeral) key pairs are generated for every session, so recorded traffic cannot be decrypted later even if a long-term key is compromised.

A plain Diffie–Hellman exchange does not authenticate the peers and is therefore vulnerable to a man-in-the-middle attack. For this reason the handshake adds HMAC-SHA-256 proofs (RFC 2104) computed with a pre-shared key over the whole handshake transcript. Only a party that knows the PSK can produce a valid proof for the exact public keys and salt used in the session.

3.3 HKDF-SHA-256

HKDF (RFC 5869) derives cryptographic keys from existing secret material using HMAC. The raw X25519 shared secret is not used directly as a key. It is passed to HKDF-SHA-256 together with the random salt and a context string (`device1-to-device2`), which produces a 32-byte key for AES-256-GCM. The context string binds the key to its purpose and direction, so a key for the opposite direction would be a different key.

3.4 AES-256-GCM and Nonce Management

AES-256-GCM (NIST SP 800-38D) is an authenticated encryption algorithm. It encrypts the plaintext and produces a 16-byte authentication tag that covers the ciphertext and optional additional authenticated data. During decryption, any change to the ciphertext, the tag, or the AAD makes the verification fail, and no plaintext is released.

The security of GCM depends critically on never using the same (key, nonce) pair twice. This project uses a deterministic counter nonce: Device1 keeps a 64-bit counter that starts at 0 for every session and is incremented before each use, so each packet, including every TCP

retransmission, receives a new value. Because each session also uses a new key, a (key, nonce) pair can never repeat. If the counter ever reached its maximum value, encryption is refused instead of wrapping around.

3.5 Linux NFQUEUE

NFQUEUE is part of the Netfilter framework. An iptables rule with the NFQUEUE target sends matching packets to a user-space application through the libnetfilter_queue library. The application receives each packet in a callback, can inspect or modify it, and returns a verdict: `NF_ACCEPT` lets the (optionally modified) packet continue through the kernel, and `NF_DROP` discards it. In this project the callback implements the encryption and decryption of the TCP payload and returns the rewritten packet with `NF_ACCEPT`.

4. Implementation

4.1 Development Environment

The project was developed on a Mac computer using a Kali Linux virtual machine running in Oracle VirtualBox. VirtualBox Shared Folders were used to exchange project files between the host and guest operating systems. Source files were edited using Visual Studio Code on macOS and stored in a shared folder named secure-channel, which was accessible inside Kali Linux at /media/sf_secure-channel. Compilation and testing were performed in the Kali Linux environment.

Table 2. *Development and test environment.*

Component	Version or configuration
Host operating system	macOS
Virtualization software	Oracle VirtualBox
Guest operating system	Kali GNU/Linux Rolling 2026.3
Linux kernel	7.1.5+kali-arm64
Architecture	ARM64 (aarch64)
Compiler	GCC 16.2.0
Cryptographic library	OpenSSL 3.6.5
Packet processing library	libnetfilter_queue 1.0.5
Firewall utility	iptables 1.8.13 (nf_tables)
Code editor	Visual Studio Code

4.2 Project Structure

Table 3. *Main files of the final implementation.*

File	Purpose
`src/device.c`	Main program. Runs the handshake, opens the NFQUEUE, and calls the packet processor for each packet (modes `encrypt` and `decrypt`).
`src/handshake.c`, `include/session_auth.h`	Authenticated X25519 handshake, HMAC proofs, HKDF key derivation.
`src/packet_processor.c`	Whole-packet logic: AAD construction, payload replacement, length update, checksum update, replay check (`pp_encrypt_packet`, `pp_decrypt_packet`).
`src/packet_crypto.c`	AES-256-GCM protection of a payload with counter nonce (`protect_payload`, `unprotect_payload`).
`src/packet_utils.c`	IPv4/TCP parsing and IP/TCP checksum recalculation.
`include/replay_window.h`	64-packet sliding anti-replay window.
`tests/unit/`	Unit tests (crypto, packet processor, replay window).
`scripts/`	`setup_topology.sh`, `protect_rules.sh`, `e2e_test.sh`.
`legacy/`	Earlier development stages (stand-alone tests and experiments); not part of the final build.

4.3 Cryptographic Implementation

All cryptography uses the OpenSSL EVP interfaces. Key pair generation and the Diffie–Hellman operation use X25519 (`EVP_PKEY_X25519`). Key derivation uses `EVP_PKEY_HKDF` with SHA-256. The packet payload is protected with `EVP_aes_256_gcm`. The handshake proofs use `HMAC` with SHA-256 and are compared with the constant-time function `CRYPTO_memcmp`. Secret values (private keys, shared secrets, session key, temporary plaintext buffers) are cleared with `OPENSSL_cleanse` when they are no longer needed, and all functions return an error code that the caller must check.

`protect_payload()` receives the session key, the packet counter, the nonce prefix, the AAD, and the plaintext. It refuses to encrypt if the counter has reached its maximum value, builds the nonce from the prefix and the counter, and outputs `counter ‖ ciphertext ‖ tag` (24 bytes longer than the plaintext). `unprotect_payload()` reads the counter, rebuilds the nonce, verifies the tag, and returns the plaintext only if the verification succeeds; on failure the output buffer is cleared. The function was checked against the AES-256-GCM known-answer test vector from the original GCM specification (Section 5.4).

4.4 Handshake and Authentication

`device_handshake()` in `src/handshake.c` implements both roles. The transcript is the concatenation of the Device1 public key, the Device2 public key, and the salt (80 bytes). Device2 sends its proof first; Device1 verifies it, sends its own proof, and Device2 verifies that. If any step fails, the function returns an error and the device exits without processing traffic. Each side then calls the same HKDF function and obtains the same 32-byte key. The session key is held in memory only and is cleared when the device stops.

4.5 NFQUEUE Packet Handler

`src/device.c` registers a callback with `nfq_create_queue()` and processes every received packet in a loop. The callback reads the packet, copies it into a work buffer (65,535 bytes plus the 24 bytes of protection overhead), and calls `pp_encrypt_packet()` in encrypt mode or `pp_decrypt_packet()` in decrypt mode. For a modified packet it calls `nfq_set_verdict()` with `NF_ACCEPT`, the new length, and the rewritten buffer; for an unchanged packet (no payload) it accepts the original; for any error it drops the packet and prints the reason (for example `AUTHENTICATION FAILED (bad tag)` or `REPLAY detected`). The program stops cleanly on SIGINT or SIGTERM and clears the key.

4.6 Packet Modification and TCP Considerations

Encryption (Device1). The packet is parsed (`parse_packet`), which rejects non-TCP packets, IP fragments, and inconsistent lengths. The 22-byte AAD is built from the header fields, the payload is protected, the result replaces the payload in the same buffer, the IP Total Length is increased by 24, and `recalculate_checksums()` recomputes the IP header checksum and the TCP checksum (including the pseudo-header). **Decryption (Device2)** performs the inverse operations: it verifies the tag over the same AAD, checks the replay window, writes the plaintext back, decreases the Total Length by 24, and recomputes both checksums. The restored packet is bit-identical to the packet sent by PC1 (verified in Section 5.4).

Sequence and acknowledgment numbers. The TCP sequence and acknowledgment numbers are not translated. PC1 and PC2 only ever see original-length segments, so the byte stream is consistent for the two TCP stacks. On the link between the devices the segments are 24 bytes longer than their sequence-number ranges, which makes consecutive segments appear to overlap in a packet capture (Figure 26). No device on that link interprets TCP state, and no stateful firewall (connection tracking) rules are used, so this does not affect the transfer.

Retransmissions and reordering. A retransmitted TCP segment is a new packet for Device1 and receives a new counter, so no nonce is ever reused. Device2 keeps a 64-packet sliding window: duplicates and packets older than the window are rejected, while reordering inside the window is tolerated. Only packets with a valid tag can move the window.

Segmentation and MTU. The PC interfaces use an MTU of 1476 (MSS 1436), so the largest protected packet is 1500 bytes. Offloading features that could hand NFQUEUE oversized packets are disabled on the virtual interfaces. **Packets without payload** (the TCP handshake, pure ACKs, FIN) are accepted unchanged on both devices, which keeps connection setup and teardown working. **Fragments** are dropped.

4.7 Anti-Replay Protection

The header `include/replay_window.h` implements a sliding window of 64 counters, in the same way as IPsec and DTLS. The receiver stores the highest counter accepted and a bitmap of the 64 preceding counters. A counter is accepted if it is higher than the highest value or lies inside the window and has not been seen before; otherwise the packet is dropped. The window is updated only after the authentication tag has been verified, so forged packets cannot manipulate it.

5. Testing and Results

The implementation was developed in stages, and each stage was tested on the Kali Linux virtual machine. Sections 5.1 to 5.3 show the results of these development-stage tests with the original screenshots: they verified the individual building blocks (key exchange, authentication, AES-GCM, replay protection, and NFQUEUE interception). Where a development-stage test differs from the final design, the difference is stated. Section 5.4 presents the tests of the final, integrated system: unit tests and an end-to-end test through all four hosts. Each test states what it verifies and what it does not.

Table 4. *Overview of the tests.*

Section	Test	Verifies
5.1	Key exchange, authentication, AES-GCM, packet crypto	Cryptographic building blocks work and reject modified data.
5.2	Device-to-device session and replay tests	Authenticated handshake, encrypted exchange, replay and tamper rejection (stand-alone programs).
5.3	NFQUEUE tests	Packets can be intercepted, modified, and reinjected; payload encryption works on intercepted packets.
5.4	Final system: unit tests	Packet-level encryption/decryption, headers, checksums, nonces, replay, fail-closed cases.
5.5	Final system: end-to-end test	Real TCP transfer PC1 → Device1 → Device2 → PC2, wire capture, forged packet.

5.1 Cryptographic Component Tests

X25519 key exchange and HKDF-SHA-256

Objective: show that both devices derive the same X25519 shared secret and the same HKDF session key. **Result:** both checks pass (Figure 4).

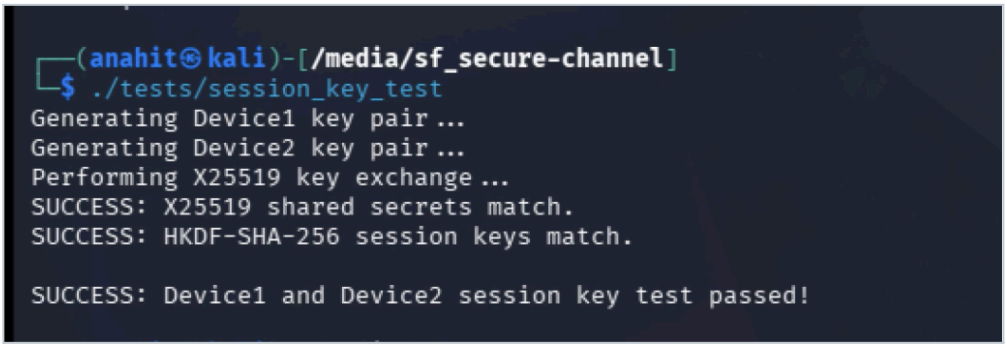


Figure 4. *X25519 shared secrets and HKDF-SHA-256 session keys match on Device1 and Device2.*

Authentication with the pre-shared key

Objective: verify that both devices accept a correct PSK proof and that the derived keys correspond. **Result:** both devices are authenticated and the derived keys match (Figure 5). This early test derived keys for both directions; the final system uses only the `device1-to-device2` key because only the PC1 → PC2 direction is protected.



Figure 5. *Authenticated session test: X25519, PSK proofs, and key derivation.*

AES-256-GCM encryption and decryption

Objective: verify encryption, decryption, tamper detection, and rejection of a wrong key. **Result:** the original payload “HELLO” is recovered after decryption, the modified ciphertext is rejected, and decryption with a wrong key fails (Figure 6).

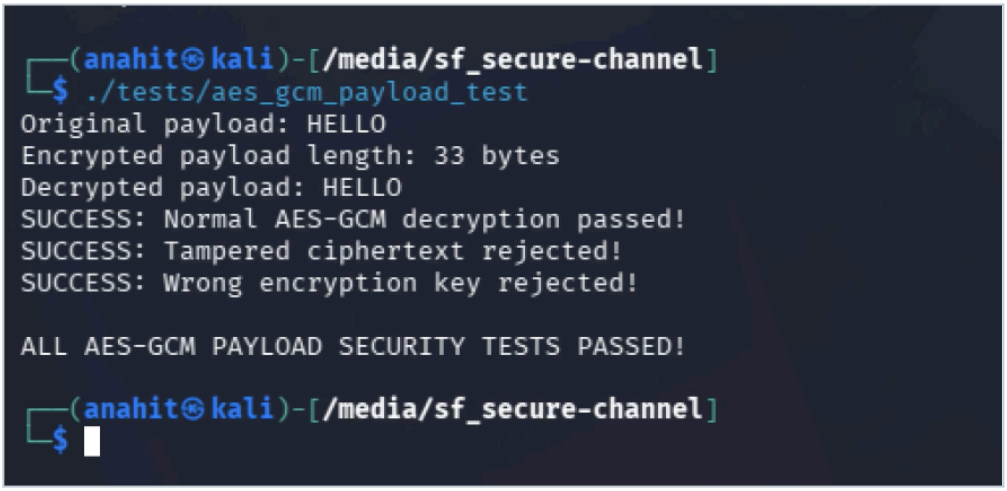


Figure 6. *AES-GCM payload test: normal decryption, tampered ciphertext rejected, wrong key rejected.*

Packet payload protection

Objective: verify payload protection with different payload types and sizes. **Result:** text, binary, and 1024-byte payloads are recovered correctly, and modified ciphertext and an incorrect key are rejected (Figure 7). Figure 8 shows the combined Device1 → Device2 flow: encryption on Device1, successful authentication and decryption on Device2.



Figure 7. *Packet payload encryption tests for different payload sizes, with tamper and wrong-key checks.*

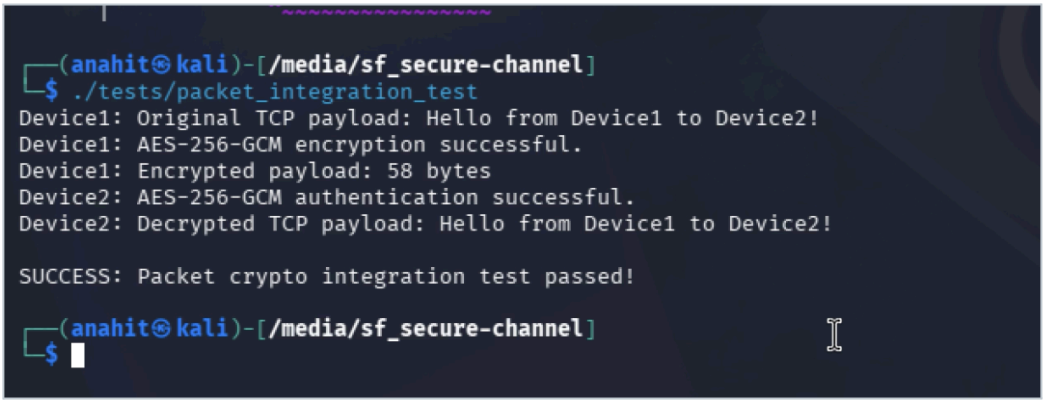


Figure 8. *Integration of the packet crypto functions: Device1 encrypts, Device2 authenticates and decrypts.*

5.2 Device-to-Device Session and Replay Tests

These stand-alone programs exchanged data over TCP between a Device1 process and a Device2 process on the loopback interface. **Loopback is not a multi-host test**; the tests verify the protocol logic, not the network path.

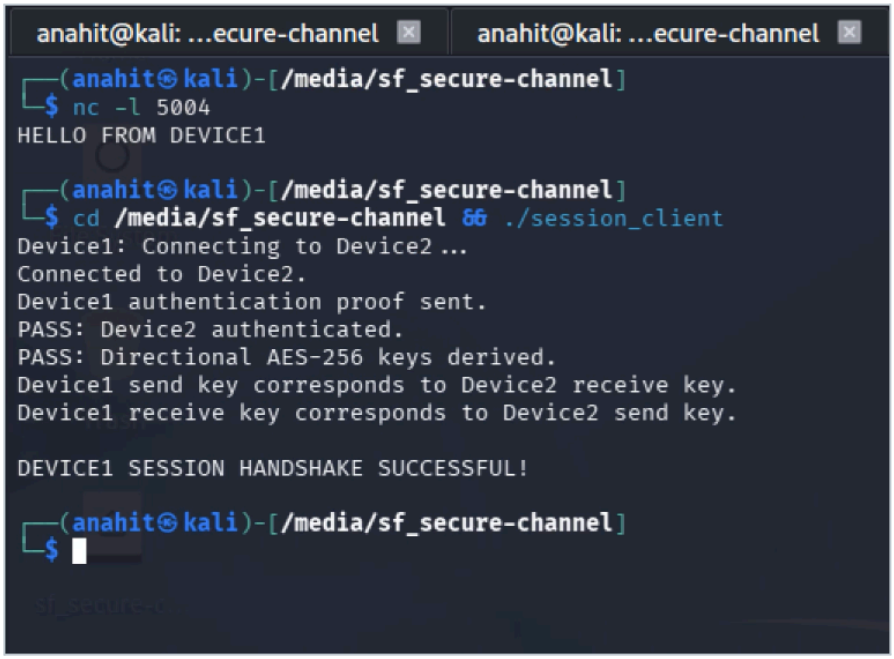


Figure 9. *Device1 side of the authenticated handshake: Device2 is authenticated and the session keys are derived.*



Figure 10. *Device2 side of the authenticated handshake.*



Figure 11. *Device1 sends an AES-256-GCM encrypted message and verifies the authenticated response.*

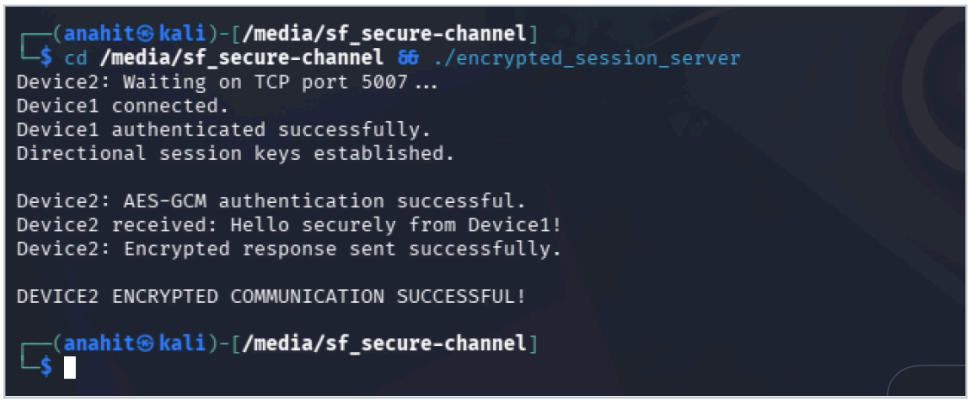


Figure 12. *Device2 authenticates and decrypts the message and sends an encrypted response.*

The replay-protection test (Figure 13) sends a normal message, a replayed message, an out-of-order message, a modified message, and a message with an incorrect key. All invalid messages are rejected. In this early prototype the receiver required strictly increasing counters, so an out-of-order message was rejected as well. The final system replaces this with a 64-packet sliding window (Section 4.7), which still rejects duplicates but tolerates the reordering that can occur on real networks. Figures 14 and 15 show a replay-protected exchange of three messages between the two device programs.

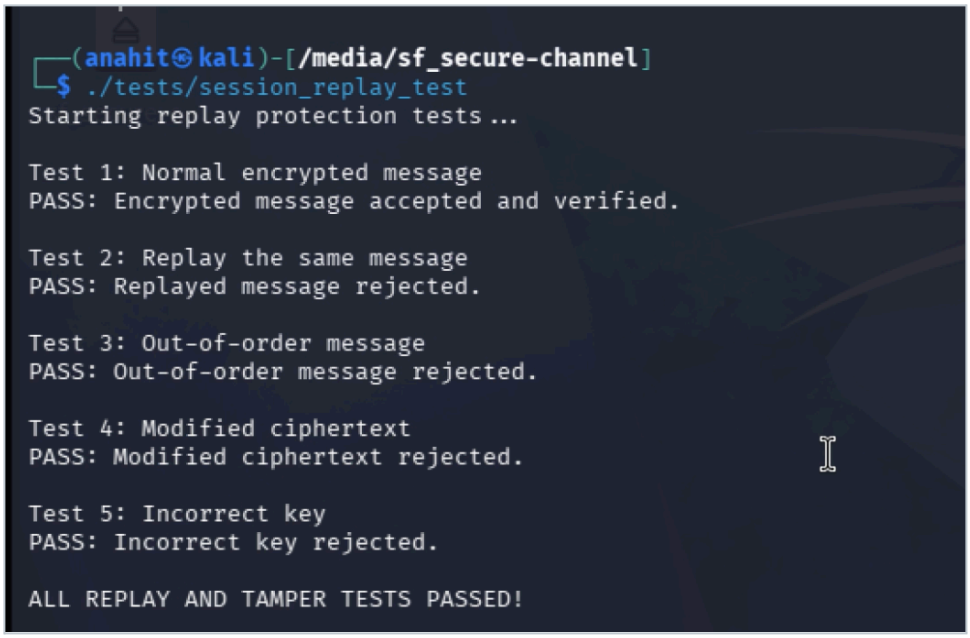


Figure 13. *Replay and tamper tests of the early prototype (strict counter order).*



Figure 14. *Device1 sends three encrypted messages and receives authenticated responses.*

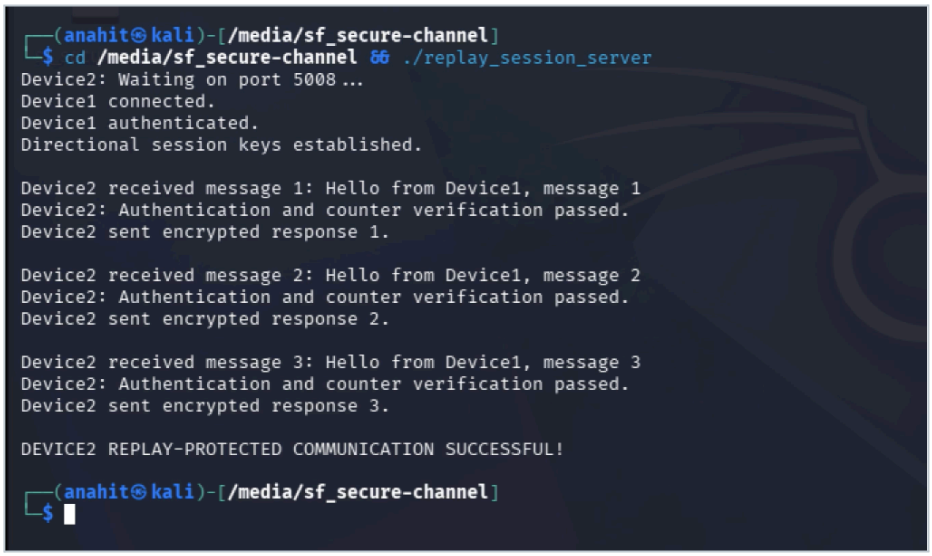


Figure 15. *Device2 verifies authentication and the counter of each message.*

5.3 NFQUEUE Tests

These tests used the loopback interface and an iptables rule with the NFQUEUE target. They verified, step by step, that packets can be received in user space, modified, reinjected, and combined with AES-GCM.

Packet inspection and modification

Objective: show selected packets reaching the user-space handler and being modified. **Result:** the handler prints packet length, ports, and payload length. For the five-byte payload it replaces “HELLO” by “WORLD” (same length, so no length change is needed yet) and returns `NF_ACCEPT`; the receiving server prints “WORLD” (Figures 16 and 17). This proved that a modified packet is accepted by the kernel and delivered to the application.

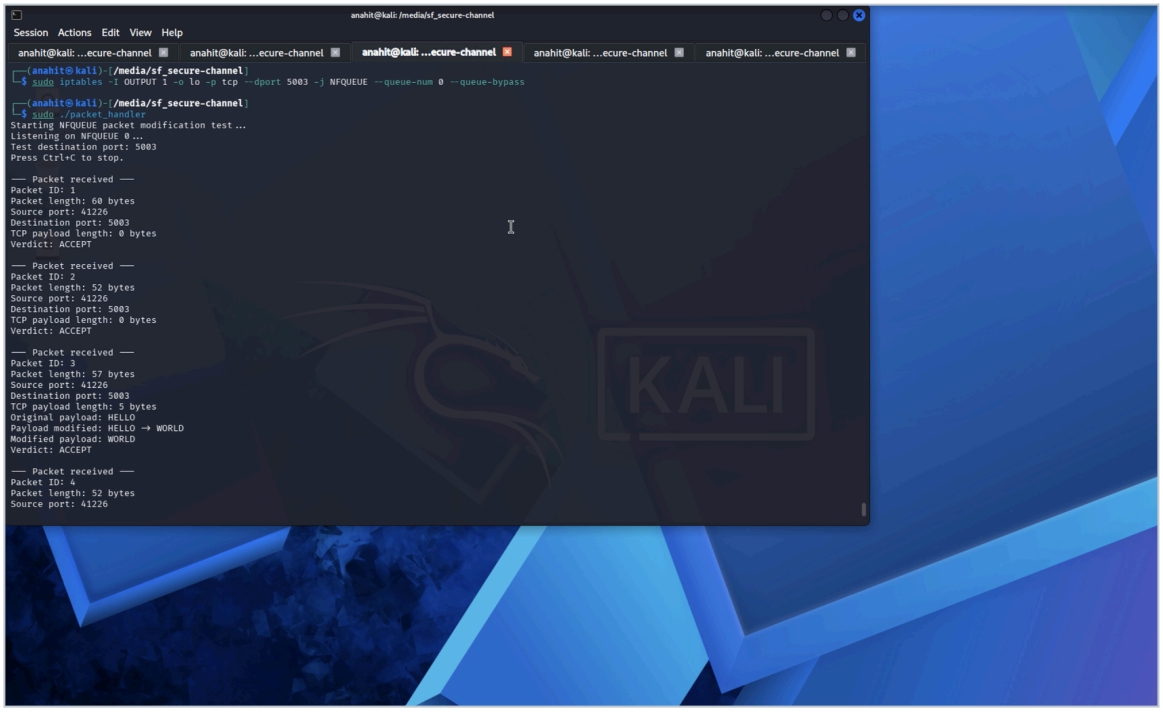


Figure 16. *NFQUEUE handler: packet fields are printed and the payload is modified before the verdict.*

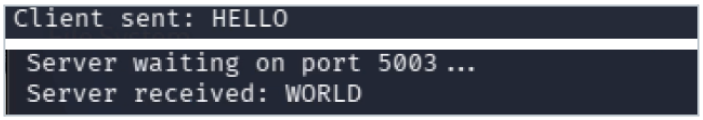


Figure 17. *The client sends “HELLO”; the server receives the modified payload “WORLD”.*

AES-GCM on intercepted packets

Objective: connect NFQUEUE with the packet crypto functions. **Result:** an intercepted 19-byte TCP payload becomes a 47-byte AES-GCM payload and is decrypted and verified successfully (Figure 20). In this early version the nonce was sent in full (12 + 16 = 28 bytes overhead); the final design sends only an 8-byte counter, giving 24 bytes of overhead. In the NFQUEUE tests shown in Figures 18 and 21 the packet itself was still accepted unchanged, i.e. the encryption was verified on a copy of the payload but not yet applied to the forwarded packet. Applying it to the forwarded packet is what the final system adds.

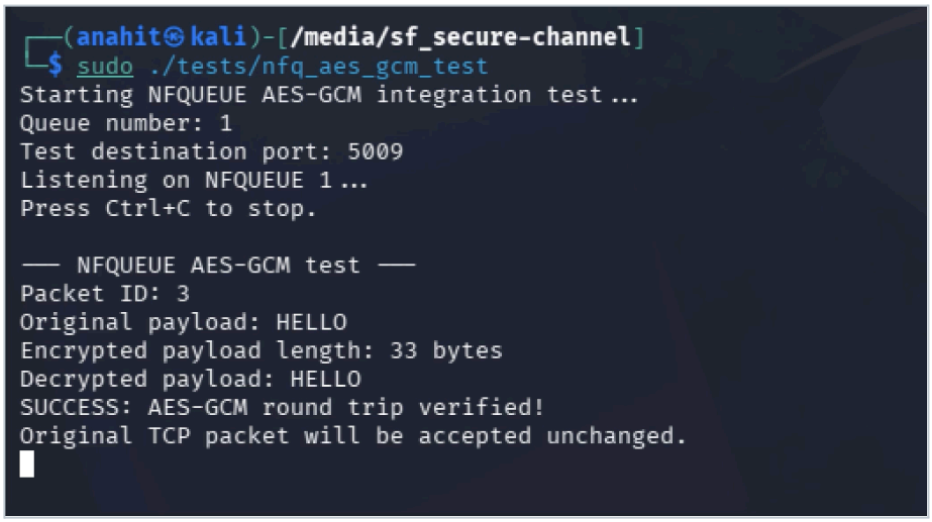


Figure 18. *NFQUEUE AES-GCM round-trip test on an intercepted payload (original packet accepted unchanged).*



Figure 19. *Client sends "HELLO"; the server receives the unchanged "HELLO" in this test.*

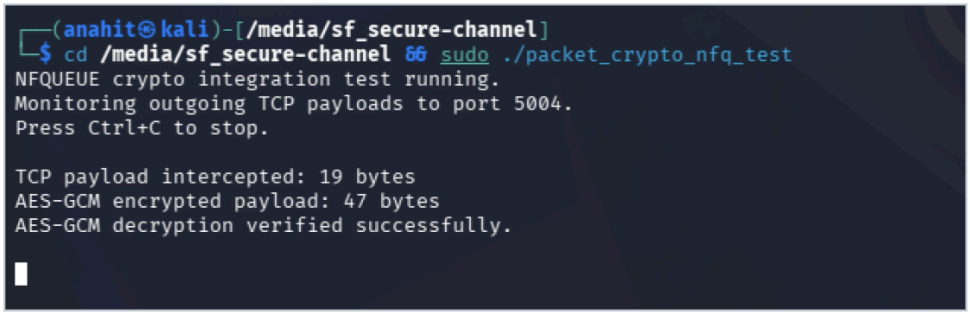


Figure 20. *Interception of a 19-byte payload, encryption to 47 bytes, and successful verification.*



Figure 21. *Authenticated session established before the NFQUEUE test starts.*

Device modes with the authenticated handshake

In the next stage the device program ran the authenticated handshake first and then opened the queue, in encrypt mode (queue 2) and decrypt mode (queue 3). The packets were only inspected and accepted (Figures 22 and 23); this stage proved the handshake-then-queue sequence that the final program uses.

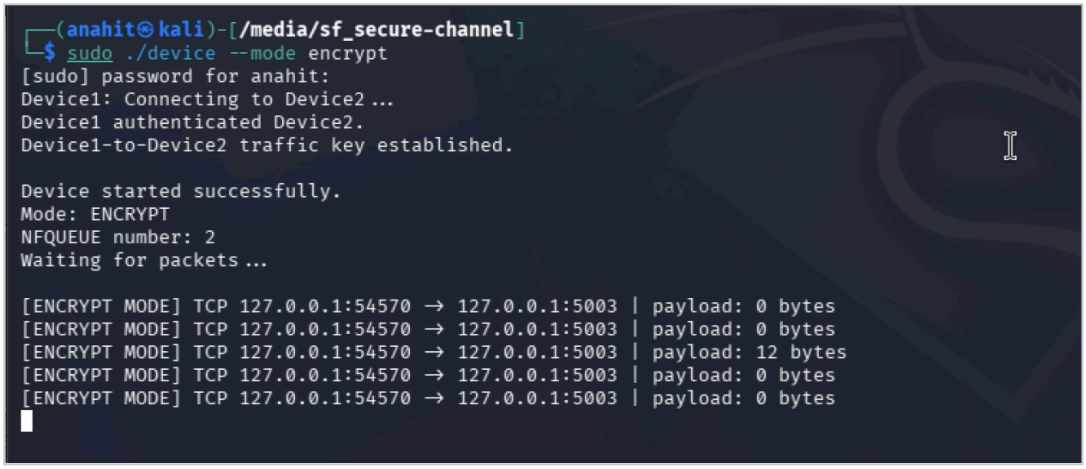


Figure 22. *Device1 (encrypt mode): handshake completed, packets received on queue 2 (inspection stage).*

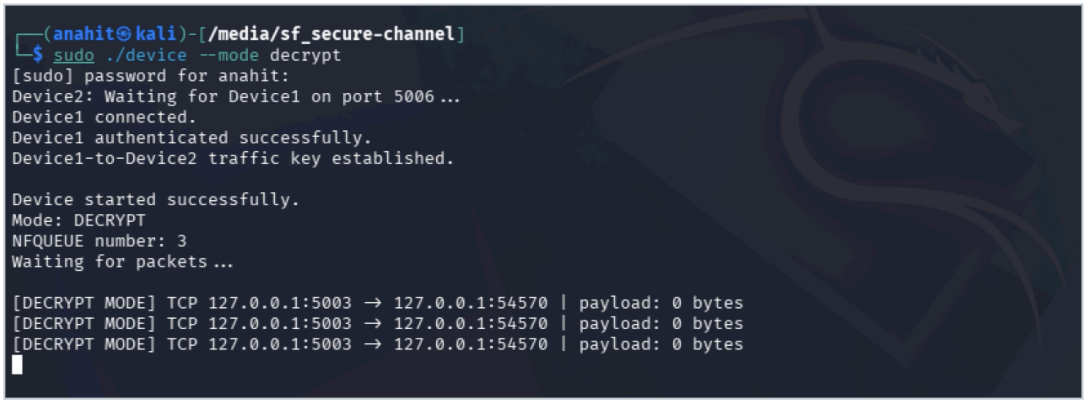


Figure 23. *Device2 (decrypt mode): Device1 authenticated, packets received on queue 3 (inspection stage).*

5.4 Final System: Unit Tests

The final packet processor was tested without root privileges and without netfilter, using packets built in memory. The command `make test` builds and runs three programs. The tests verify the following:

- **AES-256-GCM known-answer test:** the output for the NIST/McGrew–Viega test case 14 (all-zero key and nonce) matches the published ciphertext and tag. Changing any single bit of the counter, ciphertext, tag, AAD, or key causes rejection; an exhausted counter makes encryption fail.
- **Round trip:** after encryption and decryption the packet is bit-identical to the original, with and without TCP options. The IPv4 and TCP checksums are validated by an independent implementation of the checksum algorithm, not by the code under test.
- **Header preservation and confidentiality:** addresses, ports, sequence and acknowledgment numbers, flags, and options are unchanged after encryption; the plaintext does not appear in the output; the packet length grows by exactly 24 bytes.
- **Unique nonces:** 5000 identical packets produce 5000 different, strictly increasing counters and different ciphertexts.
- **Tamper and replay:** a change to any byte of the counter, ciphertext, or tag, or to the address, port, sequence, acknowledgment, or flag fields, is rejected. A replayed packet is rejected. Forged packets do not move the replay window. Out-of-order delivery within the window is accepted. A decrement of the TTL by a router does not break authentication.
- **Fail-closed behavior:** packets with insufficient buffer space, fragments, truncated protected packets, and plaintext packets injected on the decrypt side are rejected.
- **Replay window:** in-order, duplicate, reordered, too-old, large-jump, and window-edge cases.

```
$ make && make test
== build/test_packet_crypto
packet crypto: all checks passed (incl. NIST AES-256-GCM vector)
== build/test_packet_processor
packet processor: all checks passed
== build/test_replay_window
replay window: all checks passed
ALL UNIT TESTS PASSED
```

Figure 24. *Output of `make test` on Kali Linux. The build produced no compiler warnings.*

These tests prove the logic of the packet functions. They do not prove correct behavior in a real TCP session; that is the purpose of the end-to-end test.

5.5 Final System: End-to-End Test

Objective: verify the complete data path PC1 → Device1 → Device2 → PC2 with real TCP traffic. **Procedure:** the script `scripts/e2e_test.sh` (started with `sudo make e2e`) creates the four namespaces, starts Device2 and Device1 and waits for the handshake, installs the iptables NFQUEUE rules for destination port 5001, captures traffic on the Device1 → Device2 link and at PC2 with tcpdump, sends a file of about 3 MB from PC1 to PC2, and then evaluates the results. A second part injects a forged TCP packet with a plaintext payload directly onto the link between the devices.

```
$ sudo make e2e
== 1. Topology
Topology set up: pc1 can reach pc2.
== 2. Start devices and run the handshake
  PASS: handshake completed on both devices
== 3. Enable protection and capture
Protecting TCP 10.0.1.0/24 -> 10.0.3.0/24, destination port 5001
== 4. Send data PC1 -> PC2
== 5. Verify
  PASS: PC2 received exactly what PC1 sent (sha256 match)
  PASS: plaintext NOT visible on the Device1->Device2 link
  PASS: plaintext IS visible at PC2 (restored)
  PASS: Device1 encrypted packets
  PASS: Device2 verified tags and decrypted packets
  PASS: no packet was dropped during the normal transfer
  PASS: every nonce counter is unique (strictly increasing, no repeats)
  PASS: largest TCP payload: 1424 bytes at PC2 vs 1448 on the device link (+24 = counter 8 + GCM tag
16)
== 6. Forged packet injection (plaintext payload straight onto the device link)
  PASS: Device2 rejected the forged packet (authentication failed)
  PASS: forged data never reached PC2

Device1 log (last lines):
[ENCRYPT] counter=2296  1476 -> 1500 bytes
[ENCRYPT] counter=2297  1476 -> 1500 bytes
[ENCRYPT] counter=2298  440 -> 464 bytes
Device2 log (last lines):
[DECRYPT] tag OK  1500 -> 1476 bytes
[DECRYPT] tag OK  464 -> 440 bytes
[DROP] AUTHENTICATION FAILED (bad tag)

RESULT: 11 passed, 0 failed
Evidence saved in build/e2e/ (wire_d1_d2.pcap = encrypted link, pc2.pcap = restored)
```

Figure 25. *Output of the end-to-end test on Kali Linux (11 of 11 checks passed).*

Table 5. *Interpretation of the end-to-end checks.*

Check	What it shows
SHA-256 of sent and received data equal	The receiver obtained exactly the data sent by PC1: decryption, restoration, checksums, and TCP behavior are correct for a multi-megabyte transfer.
Plaintext absent on the device link	A marker string sent by PC1 is not found anywhere in the capture between Device1 and Device2: the payload is encrypted on the wire.
Plaintext present at PC2	The marker is found in the capture at PC2: the original data is restored before delivery.
Counters unique	The log of Device1 contains no repeated counter value (about 2,300 encrypted packets in the run shown).
Payload larger by 24 bytes	The largest TCP payload is 1448 bytes on the device link and 1424 bytes at PC2: 8-byte counter + 16-byte tag.

Check	What it shows
No drops in normal traffic	No valid packet was rejected during the bulk transfer.
Forged packet rejected	A plaintext packet injected onto the device link fails authentication on Device2 and never reaches PC2.

Packet capture analysis

The two captures are the direct evidence that the headers are preserved and only the payload is encrypted. In Figure 26 the same TCP stream (10.0.1.2 → 10.0.3.2, port 5001) is shown on the link between the devices and at PC2. The addresses, ports, flags, and acknowledgment numbers are identical; the segments on the device link carry 1448 bytes instead of 1424, and their sequence ranges (1:1449, 1425:2873, …) overlap because the sequence number advances only by the original length. At PC2 the ranges are contiguous (1:1425, 1425:2849, …).

```
Link Device1 -> Device2 (encrypted):
10.0.1.2.59400 > 10.0.3.2.5001: Flags [S], seq 1940863861, length 0
10.0.3.2.5001 > 10.0.1.2.59400: Flags [S.], seq 1768583740, ack 1940863862, length 0
10.0.1.2.59400 > 10.0.3.2.5001: Flags [.], ack 1, length 0
  ...
10.0.1.2.59400 > 10.0.3.2.5001: Flags [.], seq 1:1449, ack 1, length 1448
10.0.1.2.59400 > 10.0.3.2.5001: Flags [.], seq 1425:2873, ack 1, length 1448
10.0.1.2.59400 > 10.0.3.2.5001: Flags [.], seq 2849:4297, ack 1, length 1448
10.0.1.2.59400 > 10.0.3.2.5001: Flags [.], seq 4273:5721, ack 1, length 1448

At PC2 (restored):
10.0.1.2.59400 > 10.0.3.2.5001: Flags [S], seq 1940863861, length 0
10.0.3.2.5001 > 10.0.1.2.59400: Flags [S.], seq 1768583740, ack 1940863862, length 0
10.0.1.2.59400 > 10.0.3.2.5001: Flags [.], ack 1, length 0
  ...
10.0.1.2.59400 > 10.0.3.2.5001: Flags [.], seq 1:1425, ack 1, length 1424
10.0.3.2.5001 > 10.0.1.2.59400: Flags [.], ack 1425, length 0
10.0.1.2.59400 > 10.0.3.2.5001: Flags [.], seq 1425:2849, ack 1, length 1424
10.0.3.2.5001 > 10.0.1.2.59400: Flags [.], ack 2849, length 0
10.0.1.2.59400 > 10.0.3.2.5001: Flags [.], seq 2849:4273, ack 1, length 1424
```

Figure 26. *tcpdump listing of the same TCP stream on the device link and at PC2 (from the capture files saved by the end-to-end test).*

Figure 27 compares the beginning of the first data packet in both captures. On the device link the payload appears as random bytes; at PC2 the marker text sent by PC1 is readable.

```
Device1 -> Device2 (payload = counter + ciphertext + tag):
  0x0020:  8010 007f 4ca1 0000 0101 080a d7d5 b720  ....L...........
  0x0030:  a563 79f0 0000 0000 0000 0000 9d4f ff4b  .cy.........O.K
  0x0040:  d47b f313 2db0 9dfa 5f1d 3100 1d2f c74d  .{..-..._.1../.M
  0x0050:  a0fe 890f c491 0b7b 9164 d582 1fac 9368  .......{.d.....h
  0x0060:  2675 7abc f0e9 8c70 0814 ce2a b25e 6d30  &uz....p...*.^m0

At PC2 (payload restored):
  0x0020:  8010 007f 09bf 0000 0101 080a d7d5 b720  ................
  0x0030:  a563 79f0 544f 502d 5345 4352 4554 2d50  .cy.TOP-SECRET-P
  0x0040:  4c41 494e 5445 5854 2d4d 4152 4b45 522d  LAINTEXT-MARKER-
  0x0050:  3132 3334 350a 544f 502d 5345 4352 4554  12345.TOP-SECRET
  0x0060:  2d50 4c41 494e 5445 5854 2d4d 4152 4b45  -PLAINTEXT-MARKE
```

Figure 27. *Hexadecimal dump of the first data packet: ciphertext on the device link, plaintext at PC2.*

Limits of this test: it runs in network namespaces on one virtual machine, over virtual links, with one TCP connection at a time and without packet loss or reordering. It therefore does not measure performance and does not test physical networks. Retransmission and reordering behavior is covered by the unit tests but not by this end-to-end run.

5.6 Requirements Summary

Table 6. *Summary of the requirements and the evidence.*

Requirement	How it is met	Evidence
Inline path PC1 → Device1 → Device2 → PC2	Four namespaces; Device1 and Device2 are routers with NFQUEUE.	Fig. 1, 5.5
Selected TCP/IPv4 traffic	iptables rule: TCP, 10.0.1.0/24 → 10.0.3.0/24, destination port.	5.5
Secure session before data	Handshake first; queue opened only after both proofs verify.	Fig. 3, 5.2
Diffie–Hellman key exchange	Ephemeral X25519 plus HKDF-SHA-256, authenticated by HMAC proofs.	Fig. 4, 5
Encrypt only the TCP payload with AES-GCM	AES-256-GCM on the payload; headers untouched; tag appended.	5.4, Fig. 26, 27
No UDP encapsulation or custom header	The packet stays TCP/IPv4 with original addresses and ports.	Fig. 26
Verify tag, decrypt, restore, checksums	`pp_decrypt_packet()`; restored packet is bit-identical.	5.4, 5.5
Unique nonce per packet	64-bit counter incremented before every use; new key per session.	5.4, 5.5

6. Security Analysis and Limitations

- **Peer authentication:** a plain Diffie–Hellman exchange would be vulnerable to a man-in-the-middle attack. The handshake therefore requires HMAC-SHA-256 proofs over the complete transcript with a pre-shared key. The limitation is that the PSK is hard-coded in the source code for demonstration; anyone who obtains it can impersonate a device. The PSK only authenticates the handshake: because the session keys come from ephemeral X25519 keys, a later disclosure of the PSK does not reveal the keys of past sessions.
- **Nonce management:** each packet uses a nonce made of a fixed prefix and a counter that is incremented before every use and is never reset within a session. Each session has a new key. Encryption stops when the counter would overflow. The risk that remains is an implementation error or a restart of the device with an old key; a restart performs a new handshake and therefore uses a new key.
- **Directional keys:** only one direction (PC1 → PC2) is protected, and a single key labeled `device1-to-device2` is used for it. If both directions were protected, a second key and a separate counter space would be required.
- **Integrity and replay protection:** any modification of the ciphertext, counter, tag, or authenticated header fields is detected and the packet is dropped. Replays are rejected by a 64-packet window. Not authenticated are fields that routers may change or that are not security relevant for the payload, such as TTL, TOS, IP identification, the TCP window, TCP options, and the urgent pointer; an attacker on the link could modify these without detection (for example, the TCP window).
- **Confidentiality limits:** the IP and TCP headers and the packet sizes are visible. The payload length is revealed (constant +24 bytes). The system does not hide who communicates with whom, or traffic volumes.
- **Availability:** an attacker on the device link can inject or drop packets. Injected packets are rejected after authentication, but they still consume processing time. The handshake port is not protected against connection flooding.
- **TCP semantics:** the length change is handled by restoring the original length on Device2. Fragmented packets and IP options beyond the supported header parsing are not processed; fragments are dropped. The MTU of the PC interfaces must leave room for the 24 bytes of overhead.
- **Operational scope:** the tests ran in network namespaces on one virtual machine. They were not run across separate virtual machines or physical hosts, and they do not include packet loss, reordering, or multiple parallel connections.
- **Not production-ready:** the prototype has no re-keying, no secure key provisioning, no IPv6 support, no privilege separation, and no formal security review.

7. Conclusion

This project implemented a Linux-based prototype of a secure inline communication channel between two hosts through two devices: PC1 → Device1 → Device2 → PC2. The devices establish a secure session before any protected data is sent, using an authenticated X25519 Diffie–Hellman exchange and HKDF-SHA-256. For selected TCP/IPv4 packets, Device1 encrypts only the TCP payload with AES-256-GCM, keeps the original IP and TCP headers, and appends the authentication tag. The packet is forwarded directly to Device2 without UDP encapsulation or any custom header. Device2 verifies the tag, decrypts the payload, restores the original packet, recalculates the IP and TCP checksums, and forwards it to PC2. A unique nonce is used for every packet, and replayed, modified, or forged packets are dropped.

The implementation was verified in several ways: cryptographic component tests, an AES-256-GCM known-answer vector, packet-level unit tests, and an end-to-end test over four network namespaces. In the end-to-end test, a transfer of about 3 MB arrived at PC2 bit-identical; the capture between the devices contained no plaintext; every protected packet was exactly 24 bytes larger; all counters were unique; and a forged packet was rejected.

The main limitations are that only one direction is protected, the pre-shared key is hard-coded, there is no re-keying, and the tests ran in virtual namespaces on a single machine. These limitations are documented in Chapter 6 and form the basis of the future work in Chapter 7. The prototype demonstrates that the required packet transformation can be implemented in user space with Linux NFQUEUE, while the end hosts continue to use ordinary TCP.

Appendix A. Build and Execution Commands

Working directory: the project folder (for example /media/sf_secure-channel or a copy in the home directory). Root privileges are required for the end-to-end test and for running the devices.

Dependencies (Kali Linux)

```
sudo apt install build-essential libssl-dev libnetfilter-queue-dev \
                 iptables iproute2 ethtool tcpdump python3 iputils-ping
```

Build and test

```
make            # builds build/device
make test       # unit tests (no root needed)
sudo make e2e   # full end-to-end test in network namespaces
```

Manual demonstration

```
sudo ./scripts/setup_topology.sh                          # namespaces pc1, d1, d2, pc2
sudo ip netns exec d2 ./build/device --mode decrypt        # terminal 1 (start first)
sudo ip netns exec d1 ./build/device --mode encrypt        # terminal 2
sudo ./scripts/protect_rules.sh 5001                       # protect destination port 5001
sudo ip netns exec pc2 nc -l 10.0.3.2 5001                 # terminal 3
echo hello | sudo ip netns exec pc1 nc 10.0.3.2 5001       # terminal 4
sudo ip netns exec d1 tcpdump -A -i veth-d1-out tcp port 5001   # ciphertext on the device link
sudo ./scripts/protect_rules.sh clear                      # remove the rules
```

iptables rules installed by protect_rules.sh

```
ip netns exec d1 iptables -A FORWARD -s 10.0.1.0/24 -d 10.0.3.0/24 \
      -p tcp --dport 5001 -j NFQUEUE --queue-num 2
ip netns exec d2 iptables -A FORWARD -s 10.0.1.0/24 -d 10.0.3.0/24 \
      -p tcp --dport 5001 -j NFQUEUE --queue-num 3
```

Appendix B. Additional Test Output

The complete source code, the scripts, the unit tests, and the capture files produced by the end-to-end test (`docs/evidence/`) are available in the GitHub repository of the project (https://github.com/AnahitDarbinyan/secure-channel). The repository contains the following evidence files.

Table 7. *Evidence files in the repository.*

File	Content
`docs/evidence/e2e_test_output.txt`	Complete output of the end-to-end test.
`docs/evidence/wire_device1_to_device2_ENCRYPTED.pcap`	First packets captured on the Device1 → Device2 link (encrypted payload).
`docs/evidence/at_pc2_RESTORED.pcap`	First packets captured at PC2 (restored payload).
`docs/evidence/device1.log`, `device2.log`	Logs of the two devices (counters, tag verification, drop reasons).